Relatório de Conformidade & Gap Analysis

Comparativo do Hub Central vs. Repositório Template Oficial de Web III

Instituição: FATEC Olímpia — CST DSM **Disciplina:** Desenvolvimento Web III (2026/2)

Template Base: `web3-fatec-olimpia-2026-2/template-projeto` **Data:** 25/10/2026

Visão Executiva: O nosso projeto já atingiu um nível de excelência muito acima da média acadêmica em arquitetura de backend, isolamento multi-tenant, segurança criptográfica e testes (213 testes 100% verdes). No entanto, o repositório base da disciplina impõe **artefatos obrigatórios de governança**, **CI/CD no GitHub**, **dependências curriculares (DRF, NoSQL, Pytest, Ruff)** e **formato estrito de README** que ainda precisam ser incorporados para a entrega oficial.

1ª LISTA O que está CONFORME (ou supera) o Template

1. Arquitetura Modular Django e Bounded Contexts — SUPERA

O projeto está estruturado em apps independentes sob `apps/` (`accounts`, `anuncios`, `catalogo`, `core`, `financeiro`, `marketplaces`, `pedidos`, `site`, `tenancy`), com domínio modelado e desacoplado, superando amplamente o monólito tradicional.

2. Isolamento Multi-Tenant e Matriz RBAC Completa — SUPERA

Isolamento total de banco por Loja (tenant), com hierarquia estrita de 4 perfis (`DEV`, `ADMIN`, `SUPERVISOR`, `USUARIO`), consultas blindadas por tenant e testes validando bloqueio com `403 Forbidden` para perfis não autorizados.

3. Segurança Defensiva, Criptografia e Webhooks Blindados — SUPERA

Tokens OAuth protegidos com criptografia Fernet em repouso no banco; webhook do Mercado Livre blindado com validação de assinatura criptográfica HMAC, proteção anti-replay, chaves por UUID e proteção contra IDOR/BOLA.

4. Volume e Cobertura de Testes Automatizados — CONFORME

Suíte com **213 testes automatizados** passando com **100% de sucesso**, cobrindo testes unitários, testes de integração, controle de concorrência com locks pessimistas (`select_for_update`), resiliência de webhooks e isolamento multi-tenant.

5. Caderno de Decisões Técnicas (ADRs Fundamentadas) — SUPERA

Possuímos **14 ADRs** completas em `docs/design/` documentando escolhas arquiteturais, padrões GoF (Strategy, Observer, State), concorrência e governança, atendendo ao mais alto padrão de engenharia.

6. Convenção de Commits e Rastreabilidade Git

CONFORME

O histórico do Git adota estritamente o padrão **Conventional Commits** (`feat`, `fix`, `chore`, `docs`, `test`, `refactor`), exatamente como exigido no guia de contribuição do template.

7. Governança e Atribuição de I.A.

CONFORME

100% dos 162 arquivos do repositório contam com cabeçalho padronizado de atribuição de I.A., validado por script automatizado em conformidade com o contrato pedagógico da disciplina.

8. Laboratório de Simulação e Mocks

SUPERA

Módulo exclusivo `apps.mockar_dados` e feature flags que viabilizam demonstração completa de fluxos e webhooks sem dependência de credenciais externas em ambiente acadêmico.

2ª LISTA O que NÃO ESTÁ CONFORME e Falta Implementar

1. Arquivos de Governança e Comunidade na Raiz

AUSENTE

O template exige três arquivos contratuais na raiz do repositório que atualmente não existem:

- **`CONTRIBUTING.md`:** Guia de contribuição da equipe, definição do fluxo Git, regras de PR (1 aprovação obrigatória, CI verde, squash and merge), padrões de código (Ruff), papéis Scrum rotativos a cada sprint (PO, Scrum Master, Dev Team) e a Definition of Done.
- **`SECURITY.md`:** Política de segurança oficial da disciplina (procedimento para notificação ao professor e fluxo de emergência em caso de commit acidental de segredos).
- **`LICENSE`:** Arquivo de licença MIT com o nome do grupo e a identificação da FATEC Olímpia.

2. Automação de CI/CD e Segurança no GitHub (.github/)

AUSENTE

O diretório `.github/` está ausente e é avaliado pelo professor via GitHub Actions:

- **`.github/workflows/ci.yml`:** Workflow de CI que executa a cada push/PR, roda o linter **Ruff**, executa os testes via **Pytest**, barra merges com cobertura inferior a **40%** e audita vulnerabilidades com **pip-audit**.
- **`.github/workflows/secret-scan.yml`:** Action com **Gitleaks** para detecção automática de credenciais ou chaves privadas vazadas no histórico.
- **`.github/dependabot.yml`:** Configuração semanal para abertura automática de PRs de correção de dependências vulneráveis.

3. README.md Acadêmico Oficial

DESATUALIZADO

O nosso `README.md` tem apenas 3 linhas. O template exige a estrutura acadêmica completa:

- Identificação formal: Grupo XX, Docente, FATEC Olímpia - CST DSM.
- Seção "Sobre o Projeto": Problema que resolve, Público-alvo e Organização parceira.
- Tabela da Equipe com nome completo, papel Scrum e link de GitHub de cada integrante.
- Instruções canônicas de execução local e comandos de teste com pytest.
- Quadro de Sprints (1 a 4) com status e links diretos para as User Stories no **Taiga**.
- Seção de entregas finais: URL em produção, Swagger (/api/docs/) e link do vídeo demo de 5 min.

4. Estrutura Canônica de Documentação em docs/

INCOMPLETO

O README do template aponta para 4 arquivos canônicos que devem existir em `docs/`:

- **`docs/arquitetura.md`:** Visão geral da arquitetura, C4 e topologia.
- **`docs/requisitos.md`:** Compilação de User Stories e Requisitos Funcionais.
- **`docs/decisoes.md`:** Documento-índice estruturado ligando e resumindo as nossas 14 ADRs.
- **`docs/manual-usuario.md`:** Passo a passo explicativo para o avaliador operar o sistema.

5. Dependências Curriculares do requirements.txt

PENDENTE

Nosso `requirements.txt` precisa incorporar os pacotes exigidos pelas aulas da disciplina:

- **DRF e OpenAPI (Sprint 3):** `djangorestframework`, `djangorestframework-simplejwt`, `drf-spectacular`, `django-cors-headers`, `django-filter`.
- **NoSQL (Sprint 2):** pymongo (integração com MongoDB Atlas requerida na grade).
- **Deploy / Servidor Web:** `gunicorn`, `whitenoise`, `psycopg2-binary`.
- **Testes e Qualidade:** `pytest`, `pytest-django`, `pytest-cov`, `factory-boy`, `faker`, `ruff`.

6. Runner Pytest e Trava de Cobertura de Código

PENDENTE

Atualmente rodamos os testes via `python manage.py test`. Precisamos criar o arquivo de configuração **`pytest.ini`** para que o comando `pytest --cov=. --cov-report=xml` execute os 213 testes no Django e gere o relatório exigido pelo CI do GitHub Actions com cobertura superior a 40%.

7. Alinhamento de .env.example e Expansão do .gitignore

PARCIAL

O `.env.example` precisa incluir as chaves padrão da disciplina (`DATABASE_URL`, `MONGODB_URI`, JWT e SMTP), e o `.gitignore` precisa ser ampliado para ignorar caches de testes (`.pytest_cache/`, `htmlcov/`, `.coverage`, `.ruff_cache/`), arquivos estáticos de produção e metadados de SO.

🎯 Plano de Ação Recomendado para o Grupo

Fase 1: Governança e Repositório (Imediato)

Adicionar `CONTRIBUTING.md`, `SECURITY.md`, `LICENSE` e a pasta `.github/` com os workflows de CI, Gitleaks e Dependabot.

Fase 2: Dependências e Configurações de Ambiente

Expandir o `.gitignore`, atualizar o `.env.example` e sincronizar o `requirements.txt` com os pacotes oficiais.

Fase 3: Unificação dos Testes com Pytest

Criar o `pytest.ini` e rodar a suíte via `pytest --cov` garantindo que o CI no GitHub Actions execute 100% verde.

Fase 4: Documentação Canônica e README

Preencher o `README.md` oficial no padrão FATEC Olímpia com os dados dos integrantes e estruturar os 4 documentos em `docs/`.